



ALEXANDRIA UNIVERSITY
FACULTY OF ENGINEERING
ELECTRICAL ENGINEERING DEPARTMENT

Power System Protection

Microcontroller-Based Overcurrent Relay
Implementation: Very Inverse Time Characteristic

Prepared by:

Abd El-Rahman Muhammad Saad Muhammad
ID: 19017058

Project Repository Link:

[Click here to access the Code and Details on GitHub](#)

Date: May 2026

Abstract

This report details the design and hardware implementation of a microcontroller-based digital overcurrent relay conforming to the IEC Very Inverse time-current characteristic. Overcurrent relays are critical components in power system protection, designed to isolate faults while coordinating with other protective devices. Using an Arduino Uno as the core processing unit, the system continuously monitors the load current via a ZMCT103C/PZEM current sensor. If the measured current exceeds the pre-set pickup value (I_s), the microcontroller calculates the exact tripping time based on the Time Multiplier Setting (TMS) and initiates a trip signal to a relay module. The project integrates an I2C LCD for real-time monitoring, push buttons for parameter adjustment, and a buzzer for fault annunciation. The implemented system demonstrates the feasibility and accuracy of using embedded systems for dynamic, inverse-time power system protection.

Contents

1	Introduction and Theoretical Background	3
2	Hardware System Components	4
3	System Architecture and Methodology	5
3.1	Current Sampling and Signal Processing	5
3.2	Inverse-Time Control Algorithm	5
3.3	Block Diagram	5
3.4	Software Flowchart	6
4	System Testing and Characteristics	7
5	Conclusion	8

1 Introduction and Theoretical Background

Overcurrent protection is the most widely used form of protection in radial distribution systems. An overcurrent relay operates when the load current exceeds a predefined threshold (Pickup Current, I_s). To achieve selectivity and proper coordination between primary and backup relays, Inverse Definite Minimum Time (IDMT) characteristics are utilized.

This project implements the **IEC Very Inverse** characteristic, which provides a steeper curve compared to standard inverse relays, making it highly suitable for applications where the fault current magnitude drops rapidly as the distance from the source increases.

The operating time (t) for an IEC Very Inverse relay is governed by the following standardized equation:

$$t = \frac{13.5}{\left(\frac{I}{I_s}\right) - 1} \times TMS \quad (1)$$

Where:

- t : Operating (tripping) time in seconds.
- I : Actual measured fault current.
- I_s : Pickup current setting.
- TMS : Time Multiplier Setting, used to shift the curve vertically for coordination.

2 Hardware System Components

The prototype was built using highly reliable, commercially available embedded components to form a complete data acquisition, processing, and tripping loop.

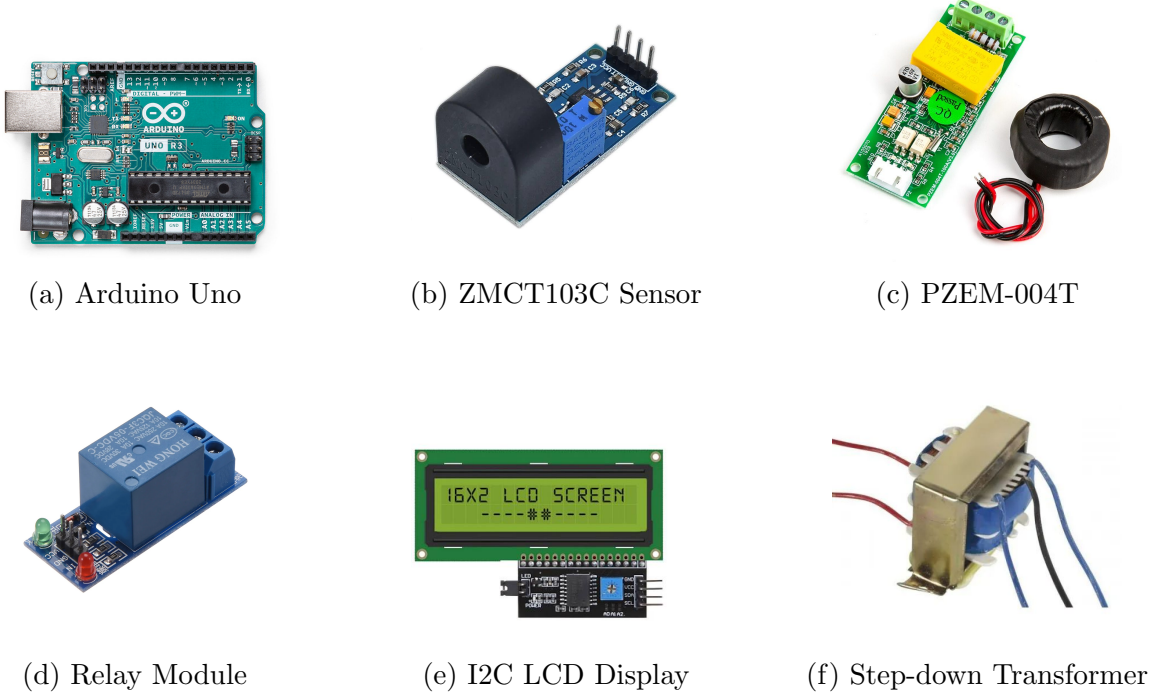


Figure 1: Primary hardware components utilized in the protective relay prototype.

- **Microcontroller (Arduino Uno):** The brain of the system, responsible for sampling the current, computing the IEC equation, and executing trip logic.
- **Current Sensing Unit:** Utilizing the ZMCT103C micro precision current transformer and PZEM-004T module to safely step down and measure the AC load current.
- **Isolation and Tripping (Relay Module):** A 1-channel 5V relay acts as the circuit breaker, physically disconnecting the load when a trip signal is issued.
- **Human-Machine Interface (HMI):** An I2C 16x2 LCD displays the real-time RMS current, calculated trip time, and system status. Push buttons are used to increment/decrement the TMS and I_s values. A buzzer provides an audible alarm upon tripping.
- **Test Load:** High-power wire-wound resistors are used to simulate varying load and fault conditions.

3 System Architecture and Methodology

3.1 Current Sampling and Signal Processing

The continuous AC current from the main line is stepped down by the current transformer. Because the Arduino's Analog-to-Digital Converter (ADC) operates strictly in the positive voltage domain (0 to 5V), the bipolar AC signal is DC-biased (offset to 2.5V).

The microcontroller samples this analog signal at a high frequency. The control algorithm calculates the True RMS value by subtracting the DC offset, squaring the discrete samples, computing the mean over a full 50 Hz AC cycle (20 ms), and finally taking the square root. This robust digital signal processing ensures high immunity to harmonics and noise, which is critical for accurate fault detection.

3.2 Inverse-Time Control Algorithm

The core logic relies on accurately executing the IEC Very Inverse characteristic. Once the calculated RMS current exceeds the user-defined pickup current (I_s), the microcontroller enters the fault-handling routine. The target tripping time (t_{trip}) is computed dynamically using Equation 1 based on the instantaneous Plug Setting Multiplier (PSM).

To handle transient or self-clearing faults, the control algorithm employs an active execution timer. If the current remains above I_s , the timer increments. If the current drops below the pickup value before the timer reaches t_{trip} , the fault is considered transient, and the timer resets to zero, preventing nuisance tripping. If the timer reaches or exceeds t_{trip} , the microcontroller instantly sends a HIGH logical signal to the relay module, opening the circuit, while activating the buzzer and displaying a "FAULT" message on the LCD.

3.3 Block Diagram

The hardware architecture follows a standard closed-loop protective relay design. As shown in Figure 2, the AC current passes through the test load and is monitored by the Current Sensor. The analog signal is fed into the Microcontroller. Based on the HMI settings (Push Buttons) and the internal logic, the Microcontroller dictates the state of the Trip Relay and updates the LCD/Buzzer.

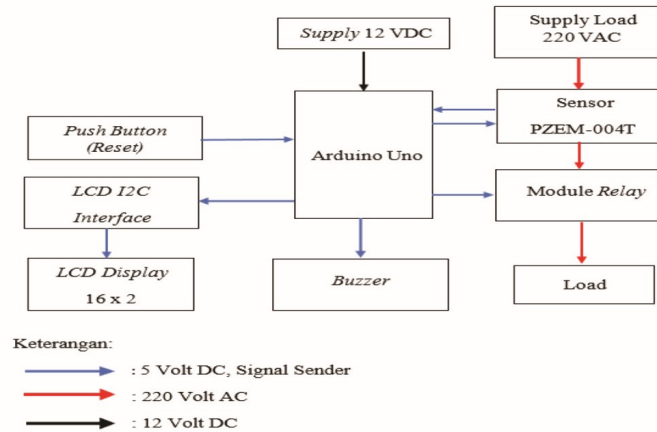


Figure 2: Block Diagram of the Microcontroller-Based Overcurrent Relay.

3.4 Software Flowchart

The software logic is designed for continuous, non-blocking monitoring. Figure 3 details the execution flow. The program continuously reads the RMS current and compares it against the Pickup Current (I_s).

If $I < I_s$, the system remains in normal operation, and the LCD displays the current. If $I \geq I_s$, a fault is detected. The microcontroller immediately calculates the required trip time using the Very Inverse formula. An internal timer starts; if the fault persists for the calculated duration, the trip signal is activated, the load is disconnected, and the buzzer sounds.

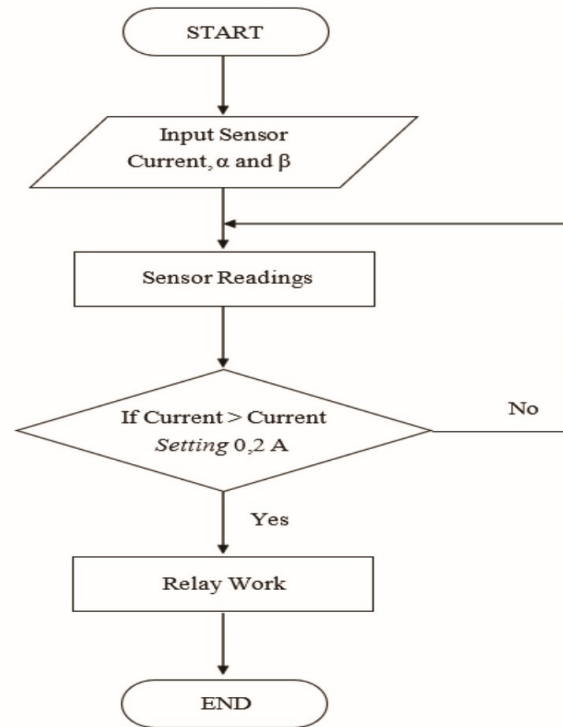


Figure 3: Software Execution Flowchart for the Inverse Time Logic.

4 System Testing and Characteristics

The relay's performance was verified by subjecting it to varying fault current levels and comparing the actual tripping time with the theoretical IEC curve.

Figure 4 illustrates the Time-Current characteristic curve plotted from the system's operational data. As the Plug Setting Multiplier ($PSM = \frac{I}{I_s}$) increases, the tripping time decreases inversely. The curve confirms that the software successfully mimics an electromechanical Very Inverse relay, providing fast fault clearance for severe short circuits while allowing temporary minor overloads (beneficial for motor starting or transient inrush currents) without nuisance tripping.

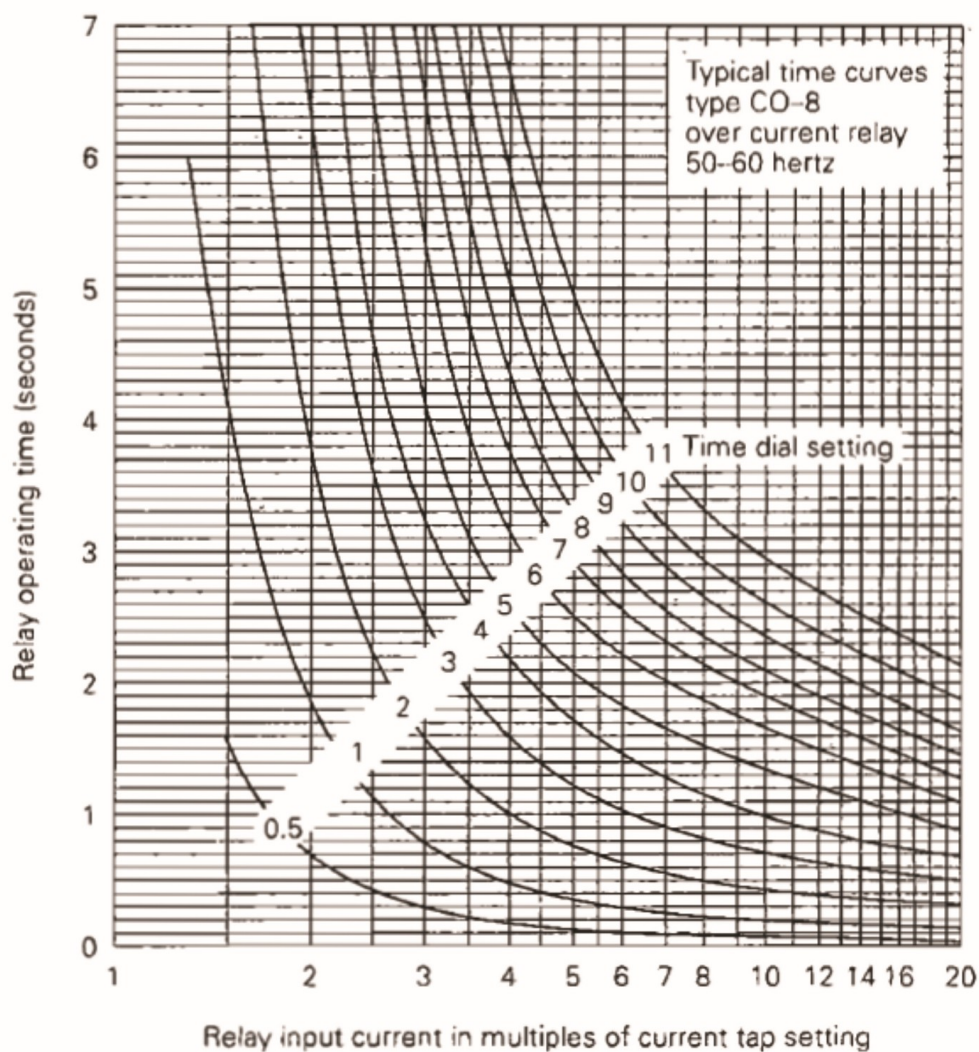


Figure 4: IEC Very Inverse Time-Current Characteristic Curve implemented in the code.

5 Conclusion

This project successfully demonstrated the design and implementation of a digital overcurrent relay utilizing an Arduino microcontroller. By integrating precise current sensing components and embedding the IEC Very Inverse time-current equation into the software, the prototype effectively protected the circuit against various overcurrent scenarios. The addition of a user-friendly HMI (LCD and push buttons) allowed for dynamic adjustment of the protection parameters (I_s and TMS), replicating the flexibility found in modern industrial numerical relays. This implementation bridges the gap between theoretical power system protection concepts and practical, low-cost embedded hardware design.